

Universal Derivative Filter REV1

Manual Transcription

This document typesets the manually reviewed transcription stored in universal_derivative_filter_REV1_manual_transcription.txt. Uncertain or illegible text is preserved as [unclear] or [illegible].

Manual transcription of `universal\_derivative\_filter\_REV1.pdf` from page PNGs.
Source pages were read manually by sub-agents without OCR.
Uncertain or illegible text is marked as `[unclear]` or `[illegible]`.

=== Page 1 ===

Before we formulate the algorithm, let's first derive the expression for the 1st - 4th order derivatives for an arbitrary point (a,b) in a coordinate system (x,y).

[figure labels: y, x, (0,0), (a,b), (x1,y1), (x2,y2), (x3,y3), (x4,y4), (x5,y5), (x6,y6)]

As shown in Fig (1) above, the line connecting origin (0,0) and (a,b) does not go through any (xi,yi), where (xi,yi) is the location of pixel center and "i" represents the "i'th" pixel surrounding (a,b).

The Taylor expansion at (a,b) can be written as

$$\begin{aligned} f(x,y) = & f(a,b) + f_x(a,b)(x-a) + f_y(a,b)(y-b) \\ & + f_{xx}(a,b)/2 (x-a)^2 + f_{xy}(a,b)(x-a)(y-b) + f_{yy}(a,b)/2 (y-b)^2 \\ & + f_{xxx}(a,b)/6 (x-a)^3 + f_{yyy}(a,b)/6 (y-b)^3 + f_{xyy}(a,b)/2 (x-a)(y-b)^2 + f_{xxy}(a,b)/2 (x-a)^2(y-b) \\ & + f_{xxxx}(a,b)/24 (x-a)^4 + f_{yyyy}(a,b)/24 (y-b)^4 + f_{xxyy}(a,b)/6 (x-a)^2(y-b)^2 + f_{xyyy}(a,b)/6 (x-a)(y-b)^3 + f_{xxyy}(a,b)/4 (x-a)^2(y-b)^2 \end{aligned} \quad (1)$$

For shorthand notation, $f(a,b)$ will be denoted by f^0
 $f_{[unclear]}(a,b)$ will be denoted by $f^0 x^3 y$ and likewise

Then Eq. (1) above can be rewritten as

$$\begin{aligned} f(x,y) = & f^0 + f_x^1(x-a) + f_y^1(y-b) + f_{xx}^2/2 (x-a)^2 + f_{xy}^2(x-a)(y-b) + f_{yy}^2/2 (y-b)^2 \\ & + f_{xxx}^3/6 (x-a)^3 + f_{yyy}^3/6 (y-b)^3 + f_{xyy}^3/2 (x-a)(y-b)^2 + f_{xxy}^3/2 (x-a)^2(y-b) \\ & + f_{xxxx}^4/24 (x-a)^4 + f_{yyyy}^4/24 (y-b)^4 + f_{xxyy}^4/6 (x-a)^2(y-b)^2 + f_{xyyy}^4/6 (x-a)(y-b)^3 + f_{xxyy}^4/4 (x-a)^2(y-b)^2 \end{aligned} \quad (2)$$

=== Page 2 ===

Note that Eq.(2) includes 0th - 4th order derivatives, and

the total number of them is 15.

Then we substitute the pixels $C(x_i, y_i)$ surrounding (a,b) into Eq.(2)

$$f(x_i, y_i) = f + f_x(x_i-a) + f_y(y_i-b) + f_{xx}(x_i-a)^2/2 + f_{xy}(x_i-a)(y_i-b) + f_{yy}(y_i-b)^2/2 \\ + f_{xxx}(x_i-a)^3/6 + f_{xxy}(x_i-a)^2(y_i-b)/2 + f_{xyy}(x_i-a)(y_i-b)^2/2 + f_{yyy}(y_i-b)^3/6 \\ + f_{xxxx}(x_i-a)^4/24 + f_{xxxy}(x_i-a)^3(y_i-b)/6 + f_{xxyy}(x_i-a)^2(y_i-b)^2/4 + f_{xyyy}(x_i-a)(y_i-b)^3/6 + f_{yyyy}(y_i-b)^4/24$$

(3)

In Eq(3) all terms of (x_i-a) and (y_i-b) are known, because they are the positions of neighbouring pixels and (a,b) [unclear]. Thus, we have 15 unknowns, which are the 0th-4th order derivatives. To obtain these 15 unknowns, we would need at least 15 pixels surrounding (a,b) , i.e. $C(x_1,y_1)$, $C(x_2,y_2)$, ..., $C(x_{15},y_{15})$.

Then we will have 15 equations according to Eq(3) they can be cast into matrix form

$$\begin{bmatrix} 1 & (x_1-a) & (y_1-b) & (x_1-a)^2/2 & (y_1-b)^2/2 & (x_1-a)(y_1-b) & (x_1-a)^3/6 & (y_1-b)^3/6 & (x_1-a)^2(y_1-b)/2 & (x_1-a)(y_1-b)^2/2 & (x_1-a)^4/24 & (y_1-b)^4/24 & (x_1-a)^3(y_1-b)/6 & (x_1-a)(y_1-b)^3/6 & (x_1-a)^2(y_1-b)^2/4 \end{bmatrix} \begin{bmatrix} f \\ f_x \\ f_y \\ f_{xx} \\ f_{xy} \\ f_{yy} \\ f_{xxx} \\ f_{xxy} \\ f_{xyy} \\ f_{yyy} \\ f_{xxxx} \\ f_{xxxy} \\ f_{xxyy} \\ f_{xyyy} \\ f_{yyyy} \end{bmatrix} = \begin{bmatrix} f(x_1,y_1) \\ f(x_2,y_2) \\ \vdots \\ f(x_{15},y_{15}) \end{bmatrix}$$

$$\begin{bmatrix} f_{xyyy} \\ f_{xxyy} \end{bmatrix} = \begin{bmatrix} . \\ f(x_{15}, y_{15}) \end{bmatrix}$$

(4)

Eq.(4): $A \in R^{(15 \times 15)}$, $z \in R^{15}$ and $b \in R^{15}$. However, if we use more than 15 pixels. Then $A \in R^{(N \times 15)}$, $z \in R^{15}$ and $b \in R^N$ where N is the number of pixels surrounding (a,b). Then Eq.(4)

=== Page 3 ===

becomes a least squares problem, and can be solved using pseudo-inverse. i.e.

$$z = (A^T A)^{-1} A^T b = A^* b. \quad (5)$$

Now we have fundamentals to calculate [unclear] order derivatives at any location and the surrounding pixels don't need to be along the horizontal or the vertical axes. Next I will apply it to the oblong filter.

The oblong filter along an arbitrary angle θ is shown on the right. The origin (a,b) is on a pixel.

(X,Y) is the global image coordinate

(x,y) is the local coordinate

The Yellow region is range of the surrounding pixels. That is, there are N pixels in the Yellow region that surround the origin (a,b).

In the global coordinate system, these surrounding pixels are

$(X_1, Y_1), (X_2, Y_2), \dots, (X_i, Y_i), \dots, (X_N, Y_N), i=1,2,\dots,N$.

First, we will convert these N surrounding pixels to the local coordinate system (x,y) associated with the oblong filter along θ direction, which is given by

$$\begin{bmatrix} x_i \\ y_i \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} X_i - a \\ Y_i - b \end{bmatrix} \quad i=1,2,\dots,N \quad (5.5)$$

Then we apply Eq. (4) to the origin. However, at origin, (a,b) in Eq. (4) will be (0,0). Then Eq. (4) for the pixel

=== Page 4 ===

(a,b) in Gk. (4) will be (0,0). Then Eq. (4) for the pixel at the origin, will be

$$\begin{bmatrix} 1 & x_1 & y_1 & x_1^2/2 & y_1^2/2 & x_1 y_1 & \dots & -x_1^2 y_1/6 & -x_1 y_1^2/6 \\ 1 & x_2 & y_2 & x_2^2/2 & y_2^2/2 & x_2 y_2 & \dots & -x_2^2 y_2/6 & -x_2 y_2^2/6 \\ \dots & & & & & & & & \\ x_m & y_m & x_m^2/2 & y_m^2/2 & x_m y_m & \dots & -x_m^2 y_m/6 & -x_m y_m^2/6 \end{bmatrix} [\text{unclear}] = \begin{bmatrix} f(x_1, y_1) \\ f(x_2, y_2) \\ \dots \\ f(x_n, y_n) \end{bmatrix} \quad (6)$$

Here subscript "0" denotes "at origin". Then 0th - 4th order derivative is written as

$$Z_0 = (A_0^T A_0)^{-1} A_0^T b_0 = [\text{unclear}] b_0 \quad (7)$$

next we will calculate the derivatives at several locations (a1,b1) (a2,b2)...(am,bm)

Here m is number of points used to resolve the half length of the filter. Again the yellow region represents the range of surrounding pixels. In addition, since (a1,b1)...(am,bm) along x axis of the local coordinate, they can be simplified as (a1,0),(a2,0)...(am,0) Select any point (ai,0) i=1,2,...m as an example we first find N surrounding pixels (x1,y1),...,(xN,yN) convert them to the local coordinate system using Eq. (5)

=== Page 5 ===

to obtain (x1,y1), (x2,y2), ... , (xn,yn). Then Eq.(4) at location (a_j,0) is given by

$$\begin{bmatrix} 1 & (x_1 - a_j) & y_1 & (x_1 - a_j)^2/2 & y_1^2/2 & \dots & 1/4(x_1 - a_j)^2 y_1^2 & 1/6(x_1 - a_j) y_1^3 \\ 1 & (x_2 - a_j) & y_2 & (x_2 - a_j)^2/2 & y_2^2/2 & \dots & 1/4(x_2 - a_j)^2 y_2^2 & 1/6(x_2 - a_j) y_2^3 \\ . \\ . \\ . \end{bmatrix}$$

```

1 (xn-a_j) yn (xn-a_j)^2/2 yn^2/2 ... 1/4(xn-a_j)^2 yn^2 1/6(xn-a_j) yn^3 ]
[unclear] = [ f(x1,y1)
               f(x2,y2)
               .
               .
               .
               f(xn,yn) ]

```

Then the nth-order derivatives at $(a_j, 0)$ location within the local coordinate system is given by

$$\bar{z}_j = (A_j^T A_j)^{-1} A_j^T b_j. \quad (9)$$

we can repeat such calculation at a_j points. i.e. z_j

Finally all these z_j will be weight-averaged to obtain the value at the origin of the local coordinate system as we did for the horizontal, vertical and 45-135-deg direction. i.e.,

$$\bar{Z}_0 = \sum_{j=1}^n W_j z_j. \quad (10)$$

where W_j is the weight applied at $(a_j, 0)$ to contour.

With the formulations above, next everything will be put together for "Numerical Implementation". The entire process consists of

=== Page 6 ===

two stages: (1) building the universal edge jitter and
(2) applying/using the jitter

For the first stage we develop the filter, we outlined

```

1. for l = 0 : 1 : 2pi/delta // [unclear] angle resolution along
   circumferential direction
   for rho = 0 : 1 : 2m // grid points (a,b) above along radial
   direction

```

```

K_j = kNN(a_j, rho) // find N surrounding points around C_j.
K_j is the indices of these N pixels.
N = length(K_j).

```

```

(X_i, Y_i) = global coordinate(C_j); // get global coordinates
of the N pixels.

```

```
(x_i, y_i) = local_coordinate(X_i, Y_i, theta); // get local coordinates of the
N pixels using Eq. (15.5)
```

```
A_j = assemble_Aj(x_i, y_i, a_j) // Assemble A_j matrix in [unclear]
A_j* = pinv(A_j) // calculate pseudo-inverse of A_j
```

(Note that not every entry in Z is needed. For example we only need df/dy , d^2f/dy^2 and d^2f/dx^2 . To reduce the memory usage, we can only save the rows of A_j^* for those entries, that is)

```
A_j^x = A_j*(g, :) // only extract rows of A_j* corresponding to
g indices. g is indices corresponding to
the derivatives of our interest.
```

```
filter{t, gridpoints}; weights = A_j^* // Save the weights in a structure
array
```

```
and for
and for:
```

* At the end of filter construction, we will have
[illegible] to the resolution of

=== Page 7 ===

```
and for
```

* At the end of filter construction, we will have
[unclear] $(2m+1)$ sets of weights, where 2δ is the resolution of
the orientation angles. $(2m+1)$ is the number of grid points along
the longitudinal direction.

The next stage is to use these sets of filter weights to calculate
the 1st - 4th order derivatives at any pixel location (a_0, b_0)
The algorithm is given below.

```
for a0 = 1 : 1 : Nl // Nl and Nu are the number of pixel
for b0 = 1 : 1 : Nu // along horizontal and vertical direction
// applicable to derivative calculations
for l = 1 : 1 : [unclear]
for j = 1 : 1 : (2m+1)
K_j = knn(a_j, 0); // extract indices of
the neighbouring
```

pixels

```
b_j = [  
    +1  
    +2  
    .  
    +N  
    ]           // collect intensity values of the  
                // N neighbouring pixels using  
                // index K_j  
  
A_j^x = filter[unclear] weights ;  
                // extract filter weights at j  
                // grid point  
  
Z_j^x = A_j^x . b_j      // calculate all derivatives.  
  
end for (j loop)  
  
Zavg(a0,b0,l) = sum(Z_j, W_j); // W_j is the weight of derivatives  
                        In Eq.(10)  
  
end for (l loop)  
  
end for (b0 loop)  
  
=== Page 8 ===  
  
end for (a0 loop)
```

* It should be noted that in this approach, we will calculate all derivatives (0th to 4th-order) in one shot. (in both normal and tangential direction)

* Another challenge is if we can avoid using "knn" function in Matlab to find N nearest surrounding pixels. We should be able to. During the filter development stage, at each [unclear] and each gridpoint (j) location, we will use "knn" to find the N nearest pixels, then we save their relative pixel distances from the origin pixel (a0, b0). For example, For j=0, we have N (x1, y1), (x2, y2), ... (xN, yN) the the relative indices for j=0 will be (x_i^{j=0}, y_i^{j=0}) - (a0, b0), which will be saved We then repeat the same for j=1 and get

$(x_i^{j=1}, y_i^{j=1}) - (a_0, b_0)$ will will be saved.

End

4/24/2022 Y. Wang @ UofSC